



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 03

NOMBRE COMPLETO: GARCÍA SITIO DIEGO

N° de Cuenta: 318120601

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 13/09/2026

CALIFICACIÓN: _____

EJERCICIO 1

Descripción:

- Se usará la proyección en perspectiva.
- Instanciar primitivas geométricas para recrear la figura central de la práctica pasada en 3D no importa que internamente se encimen los cubos. Cada pirámide de cada esquina es de un color diferente de acuerdo con los colores de las 4 pirámides de la clase pasada.
- Se requiere que exista el plano de piso negro.

Solución: Como primer paso, agregaremos todos los shaders que creamos en la práctica 2, junto con sus variables y todo lo necesario para usarlos. Un ejemplo de estos procesos es la ilustración 1. Después de tener todo lo necesario para nuestros shaders, haremos una modificación en la función del profesor para crear las pirámides triangulares, como se ve en la ilustración 2. ¿Por qué se hizo esto? Para poder hacer tetraedros de mejor manera ya que, al tener la punta afilada en el origen (0,0,0), podemos mover la pirámide de manera más sencilla que la implementación original. Como las figuras tienen ángulos de 90 grados, podemos encajarlas en el cubo de la ilustración 4 como legos sin tener que calcular rotaciones complicadas, solo usamos escalas negativas para voltear la pieza hacia adentro o hacia afuera según la esquina que queramos.

Ahora solo es implementar el suelo negro, que es un gran prisma rectangular y el cubo azul, que nos servirá para hacer la figura base de nuestro holocrón (de esta forma ya no creamos los 6 cubos azules rotados). El código de ambas figuras están en las ilustraciones 3 y 4.

```
// Mis shaders
static const char* fShaderAzul = "shaders/shader_azul.frag";
static const char* fShaderCafe = "shaders/shader_cafe.frag";
static const char* fShaderMagenta = "shaders/shader_magenta.frag";
static const char* fShaderVerde = "shaders/shader_verde.frag";
static const char* fShaderRojo = "shaders/shader_rojo.frag";
static const char* fShaderAmarillo = "shaders/shader_amarillo.frag";
static const char* fShaderNegro = "shaders/shader_negro.frag";
```

Ilustración 1: Ejemplo de uno de los pasos importantes para usar los shaders de la práctica 2.

```

94 //tetraedro rectángulo para hacer perfecta la esquina para el cubo
95 void CrearPiramideTriangular()
96 {
97     unsigned int indices_piramide_triangular[] = {
98         0, 1, 2,
99         0, 2, 3,
100         0, 3, 1,
101         1, 2, 3
102     };
103     GLfloat vertices_piramide_triangular[] = {
104         // El origen (0,0,0) es la PUNTA EXACTA de la pirámide (la esquina)
105         0.0f, 0.0f, 0.0f, // 0: Punta exterior
106         0.5f, 0.0f, 0.0f, // 1: Pata en X
107         0.0f, 0.5f, 0.0f, // 2: Pata en Y
108         0.0f, 0.0f, 0.5f // 3: Pata en Z
109     };
110     Mesh* piramidet = new Mesh();
111     piramidet->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
112     meshList.push_back(piramidet);
113 }
114
115
116
117
118
119 //Código del profesor
120 // Pirámide triangular regular
121 /*
122 void CrearPiramideTriangular()
123 {
124     unsigned int indices_piramide_triangular[] = {
125         0,1,2,
126         1,3,2,
127         3,0,2,
128         1,0,3
129     };
130     GLfloat vertices_piramide_triangular[] = {
131         -0.5f, -0.5f, 0.0f, //0
132         0.5f, -0.5f, 0.0f, //1
133         0.0f, 0.5f, -0.25f, //2
134         0.0f, -0.5f, -0.5f, //3
135     };
136     Mesh* piramidet = new Mesh();
137     piramidet->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
138     meshList.push_back(piramidet);
139 }
140 */

```

Ilustración 2: Implementación nueva para crear los triángulos triangulares.

```

// Piso negro
shaderList[8].useShader();
uniformModel = shaderList[8].getModelLocation();
uniformProjection = shaderList[8].getProjectLocation();
uniformView = shaderList[8].getViewLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.6f, -7.0f));
model = glm::scale(model, glm::vec3(10.0f, 0.5f, 10.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
// Dibujamos usando la primitiva geométrica del cubo
meshList[0]->RenderMesh();

```

Ilustración 3: Código para hacer nuestro piso tridimensional.

```

457 // El cubo azul como base (Se forma el fondo de los rombos automáticamente)
458 shaderList[2].useShader();
459 uniformModel = shaderList[2].getModelLocation();
460 uniformProjection = shaderList[2].getProjectLocation();
461 uniformView = shaderList[2].getViewLocation();
462 model = glm::scale(baseModel, glm::vec3(0.8f, 0.8f, 0.8f));
463 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
464 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
465 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
466 meshList[0]->RenderMesh();

```

Ilustración 4: Implementación del cubo azul (no tuvimos que crear varios).

```

670 //Aqui se construyen todos
671 shaderList[i].setShaderCode(
672     uniformModel = shaderList[i].getModelLocation();
673     uniformProjection = shaderList[i].getProjectionLocation();
674     uniformView = shaderList[i].getViewLocation();
675     float rSize = 0.2f; // Tamaño del punto sea
676     float rOff = 0.05f; // Desplazamiento hacia pagarlo a la cara
677     float frace y está (float en 2)
678     model = gl::translate(baseModel, gl::vec3(0.0f, 0.0f, rOff)); model = gl::rotate(model, gl::radians(45.0f), gl::vec3(0.0f, 0.0f, 1.0f));
679     model = gl::scale(model, gl::vec3(rSize, rSize, rSize));
680     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
681     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
682     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
683     model = gl::translate(baseModel, gl::vec3(0.0f, 0.0f, -rOff)); model = gl::rotate(model, gl::radians(45.0f), gl::vec3(0.0f, 0.0f, 1.0f));
684     model = gl::scale(model, gl::vec3(rSize, rSize, rSize));
685     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
686     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
687     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
688     model = gl::translate(baseModel, gl::vec3(rOff, 0.0f, 0.0f));
689     model = gl::rotate(model, gl::radians(45.0f), gl::vec3(1.0f, 0.0f, 0.0f));
690     model = gl::scale(model, gl::vec3(0.05f, rSize, rSize));
691     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
692     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
693     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
694     model = gl::translate(baseModel, gl::vec3(-rOff, 0.0f, 0.0f));
695     model = gl::rotate(model, gl::radians(45.0f), gl::vec3(1.0f, 0.0f, 0.0f));
696     model = gl::scale(model, gl::vec3(0.05f, rSize, rSize));
697     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
698     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
699     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
700     // Arriba y Abajo (Donde van V)
701     model = gl::translate(baseModel, gl::vec3(0.0f, 0.0f, rOff));
702     model = gl::rotate(model, gl::radians(45.0f), gl::vec3(0.0f, 1.0f, 0.0f));
703     model = gl::scale(model, gl::vec3(rSize, 0.05f, rSize));
704     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
705     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
706     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
707     model = gl::translate(baseModel, gl::vec3(0.0f, 0.0f, -rOff));
708     model = gl::rotate(model, gl::radians(45.0f), gl::vec3(0.0f, 1.0f, 0.0f));
709     model = gl::scale(model, gl::vec3(rSize, 0.05f, rSize));
710     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, gl::value_ptr(model));
711     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, gl::value_ptr(projection));
712     glUniformMatrix4fv(uniformView, 1, GL_FALSE, gl::value_ptr(view)); meshList[i] = ShaderMesh();
713     meshList[i] = ShaderMesh();

```

```

510 //PIRAMIDES LAS 8 ESQUINAS
511 float c = 0.4001f; // Distancia exacta a las esquinas del cubo azul
512 float p = 0.8f; // Este valor hace que la pirámide tape exactamente hasta la mitad de las aristas
513 // 4 ESQUINAS SUPERIORES (+v)
514 shaderList[7].useShader();
515 uniformModel = shaderList[7].getModelLocation();
516 uniformProjection = shaderList[7].getProjectLocation();
517 uniformView = shaderList[7].getViewLocation();
518 model = glm::translate(baseModel, glm::vec3(-c, c, c));
519 model = glm::scale(model, glm::vec3(p, -p, -p));
520 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
521 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
522 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
523 meshList[1]-->RenderMesh();
524 shaderList[6].useShader();
525 uniformModel = shaderList[6].getModelLocation();
526 uniformProjection = shaderList[6].getProjectLocation();
527 uniformView = shaderList[6].getViewLocation();
528 model = glm::translate(baseModel, glm::vec3(c, c, c));
529 model = glm::scale(model, glm::vec3(-p, -p, p));
530 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
531 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
532 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
533 meshList[1]-->RenderMesh();
534 shaderList[4].useShader();
535 uniformModel = shaderList[4].getModelLocation();
536 uniformProjection = shaderList[4].getProjectLocation();
537 uniformView = shaderList[4].getViewLocation();
538 model = glm::translate(baseModel, glm::vec3(-c, c, -c));
539 model = glm::scale(model, glm::vec3(p, -p, p));
540 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
541 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
542 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
543 meshList[1]-->RenderMesh();
544 shaderList[5].useShader();
545 uniformModel = shaderList[5].getModelLocation();
546 uniformProjection = shaderList[5].getProjectLocation();
547 uniformView = shaderList[5].getViewLocation();
548 model = glm::translate(baseModel, glm::vec3(c, c, -c));
549 model = glm::scale(model, glm::vec3(-p, -p, p));
550 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
551 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
552 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
553 meshList[1]-->RenderMesh();
554 shaderList[3].useShader();
555 uniformModel = shaderList[3].getModelLocation();
556 uniformProjection = shaderList[3].getProjectLocation();
557 uniformView = shaderList[3].getViewLocation();
558 model = glm::translate(baseModel, glm::vec3(-c, c, c));
559 model = glm::scale(model, glm::vec3(p, -p, -p));
560 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
561 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
562 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));

```

Ilustración 6: Una sección importante del código de las pirámides triangulares.

Para concluir, me pareció un ejercicio interesante y divertido, creo que el permitir que podamos observar la figura desde diferentes ángulos ayuda a entender mejor cómo mover las “piezas” para poder encajarlas a su posición correcta. Lo que me resultó curioso de preguntar cómo podía hacer este ejercicio, fue la propuesta de crear un solo cubo base y de modificar la función que el profesor creó para hacer las pirámides triangulares (sobre todo esto último, ya que modificarla hacía que la propuesta fuera más interesante). Ya después de terminar la implementación, estuve jugando con la ejecución del programa, como se puede observar en las ilustraciones 7, 8 y 9.

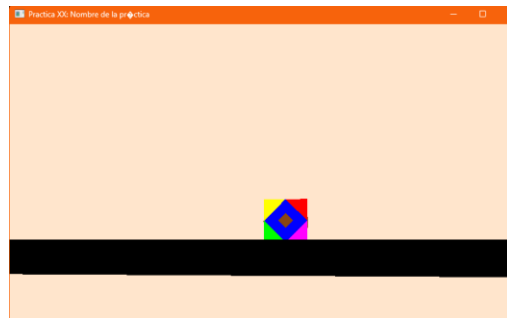


Ilustración 7: Ejecución de prueba 1.

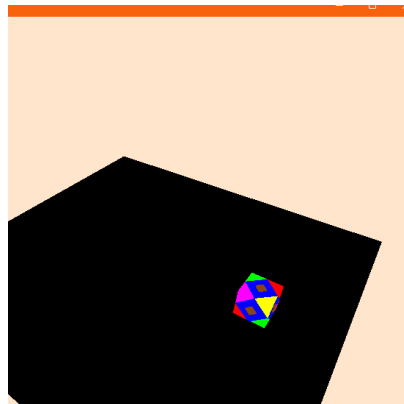


Ilustración 8: Ejecución de prueba 2.

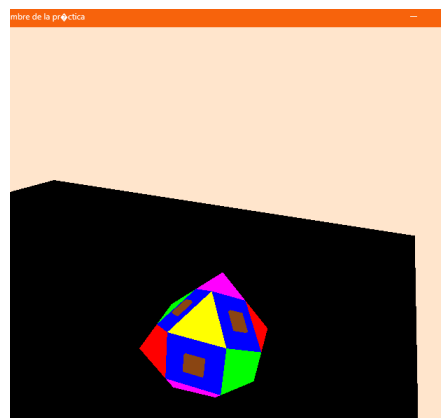


Ilustración 9: Ejecución de prueba 3